Análise de arquivo para detecção de malware

Share

codigo.py
9 lines

PY

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Decidindo como tratar a amostra, sem executá-la
Vou analisar de forma estática: decodifico as camadas de ofuscação sem executar nada do conteúdo.

Ran a command
O payload é um script Python completo, com comentários em russo. Vou ler o conteúdo inteiro, sem executar, para entender o que ele faz.

Read a file, ran a command, searched the web
A busca pelo endereço retornou só ruído sem relação. Vou tentar uma segunda busca, agora pela técnica (dead drop em memo da Solana + Node.js baixado por Python).

Searched the web
Já adianto: **não execute esse arquivo, ele é malware.** Fiz a análise de forma estática, decodificando as camadas sem executar nada.

### 1. A camada externa (o arquivo enviado)

O arquivo tem 6.558 bytes e quase nenhum código legível:

- Os nomes de variáveis são aleatórios (`aqgqzxkfjzbdnhz`, `wogyjaaijwqbpxe`…). Os módulos são carregados com `__import__('base64')` e `__import__('zlib')` em vez de `import`. É ofuscação automática, feita para dificultar a leitura e a detecção.
- Existe um bloco base64 de cerca de 6 KB. Ele é decodificado, descompactado (zlib), passado por um XOR byte a byte com a chave 134 e entregue a `exec(compile(...))`.

Esse padrão de esconder o código e executá-lo em memória é o de um *packer/loader*. O conteúdo escondido, visto a seguir, mostra por que ele existe.

### 2. Desofuscação

Refiz as três transformações (base64 → zlib → XOR 134), sem o `exec`. O resultado é um script Python de 438 linhas (15.554 bytes), com comentários em russo. Esse é o payload real.

### 3. O que o payload faz, em ordem

1. **Espera 10 segundos**, uma evasão simples contra sandboxes.
2. **Geofencing:** verifica idioma, fuso horário e deslocamento UTC. Se o sistema parecer russo, encerra em silêncio. Isso é típico de malware de autores russófonos que evitam a própria região, mas é um indício e não prova de autoria.
3. **Marcador** `~/init.json`: guarda uma data. Se ela tiver menos de 2 dias, o script não repete a execução.
4. **C2 via blockchain (dead drop):** tenta nove nós RPC públicos da Solana em sequência, consulta o endereço `BjVe…o8SC` e lê o campo *memo* da transação mais recente. Dali extrai um JSON com uma URL em base64. O atacante troca o servidor apenas publicando uma nova transação, então não há URL fixa no código para bloquear. O tráfego também parece legítimo.
5. **Baixa o 2º estágio:** faz GET nessa URL, enviando o sistema operacional no cabeçalho `os`. A resposta traz JavaScript em base64 e, nos cabeçalhos HTTP, uma chave e um IV de criptografia. Os nomes desses cabeçalhos (`ivbase64` e `secretkey`) estão escondidos em base64 no código.
6. **Baixa o Node.js v22.9.0** do nodejs.org para a sua pasta pessoal (Windows, macOS ou Linux), para ter um interpretador JavaScript próprio.
7. **Executa o código remoto:** grava um `i.js` ao lado do script, com a chave, o IV e um `eval(atob(...))` do que recebeu. Depois o roda com o Node em segundo plano, sem janela e com toda a saída descartada.

Não consigo ver o JavaScript final, porque ele vem do servidor do atacante em tempo de execução. Este arquivo é o *downloader*, e o dano real depende do que o servidor entregar.

### 4. Corroboração externa

Pesquisei essas características e elas coincidem com uma campanha já documentada, a **GlassWorm / ForceMemo**.

- O malware consulta o campo memo de transações de uma carteira Solana, já associada ao GlassWorm, para obter a URL do payload. Esse endereço é o mesmo que está no seu arquivo. thehackernews
- Os relatórios descrevem os mesmos detalhes que encontrei: alternância entre nove endpoints RPC, download do Node.js v22.9.0, carga JavaScript cifrada com AES e os arquivos ~/init.json e i.js. rescana
- A StepSecurity mostra os mesmos comentários em russo no código desofuscado. stepsecurity
- Na campanha, o atacante usa tokens roubados do GitHub para fazer force-push de código malicioso nas branches principais de repositórios Python, e o payload em base64 é anexado ao final de um arquivo Python. rescana thehackernews
- Quanto ao estágio final, análises públicas do GlassWorm o descrevem como voltado a roubar credenciais de navegador e dados de carteiras de criptomoedas. socprime

Meu veredito já se sustentava só pelo comportamento do código. A correspondência com os relatórios apenas o confirma.

### Indicadores (IOCs)

| Tipo | Valor |
|---|---|
| SHA-256 do arquivo | `d29d46edd83c523a817a693748cc462c405f093aece2e2e533673e24f98d6dc1` |
| SHA-256 do payload decodificado | `1e6487b7fbb9a0ff4c1d7032bfecd153fa6b9cc3c523243e8183588a98168267` |
| Endereço Solana (dead drop) | `BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC` |
| Artefatos no disco | `~/init.json`, `i.js` (na pasta do script), `~/node-v22.9.0-<plataforma>/` |

### Veredito: malware

É um *downloader/loader* multiestágio com C2 resolvido pela blockchain. Confiança: **muito alta**. Quatro comportamentos já seriam suspeitos isoladamente, e juntos não têm explicação legítima:

1. Esconde o código real e o executa dinamicamente.
2. Desativa-se em sistemas russos.
3. Busca instruções na blockchain para escapar de bloqueios.
4. Instala um runtime e executa, em segundo plano, código remoto que o analista não consegue ver.

### Se você chegou a executá-lo

- Apenas abrir ou ler o arquivo não infecta. Rodá-lo (`python codigo.py`) sim.
- Se rodou, desconecte a máquina da rede e trate-a como comprometida. Procure `~/node-v22.9.0-*`, `~/init.json`, um `i.js` ao lado do script e processos `node` que você não iniciou. A StepSecurity recomenda procurar `node-v22*` na pasta pessoal e `i.js` em projetos clonados recentemente. stepsecurity
- A partir de outro dispositivo limpo, troque senhas e revogue sessões e tokens (GitHub, e-mail, nuvem, chaves SSH/API). Se houver carteiras de criptomoedas, mova os fundos para uma carteira nova, porque as frases-semente podem ter sido expostas.
- O ideal é reinstalar o sistema. Se for uma máquina de trabalho, avise o time de segurança.
- Se esse trecho estava no final de um arquivo de um repositório ou pacote, considere o projeto inteiro comprometido e avise os mantenedores.

Você também pode pesquisar o hash no VirusTotal para ver as detecções. Se me disser em qual sistema operacional o arquivo chegou a rodar, posso detalhar os passos de verificação para ele.

Qual é a categoria principal deste malware?

Descrevendo o papel final do carregador e seu processo de download

**Downloader (ou *loader*) de primeiro estágio**, de ataque multiestágio.

Todo o código do payload gira em torno de trazer e executar outro código. Ele descobre o servidor pelo memo na Solana, baixa o JavaScript do segundo estágio, baixa o Node.js para poder executá-lo e dispara o `i.js` em segundo plano. Não há nenhum trecho que leia navegadores, carteiras ou arquivos pessoais, nem que envie dados para fora. Por isso, por si só, ele não se classifica como infostealer, RAT ou ransomware.

A camada externa (base64 + zlib + XOR + `exec`) funciona como um *packer*, que desempacota e executa o estágio 1 em memória. Mas o que o estágio 1 faz de fato é baixar o resto, então "downloader" é a classificação mais precisa.

Características secundárias, com as técnicas MITRE ATT&CK correspondentes:

- **Ofuscação e empacotamento:** T1027 e T1140.
- **Evasão:** atraso de 10 s (T1497.003) e geofencing que evita sistemas russos (T1614.001).
- **C2 por *dead drop resolver* na blockchain:** T1102.001.
- **Transferência e execução de código remoto:** T1105 e T1059.007 (JavaScript via Node.js).

O objetivo final está no estágio seguinte, que não está no arquivo e que eu não vi. Análises públicas da campanha GlassWorm descrevem a carga final como voltada ao roubo de credenciais de navegador e de dados de carteiras de criptomoedas. Isso é inferência a partir desses relatórios e não algo que eu tenha confirmado no código analisado. socprime